Applications of Parallel Programming

Chris Kauffman

Last Updated:
Thu Apr 30 01:43:06 PM EDT 2026

Logistics

A4 Due Yesterday

- Lots of competition for GPUs: good for folks to understand the realities of using expensive hardware
- To soften the blow, A4 Late penalties adjusted:

-5%	1 Day Late
-10%	2 Days Late

Endgame

Date	Event
Wed 29-Apr	A4 Due
Thu 30-Apr	ME4, Applications
Tue 05-May	Applications
Thu 07-May	Finale
Thu 14-May	Final Exam 10:30am-12:30pm

Course Experiences Now Open

e.g. Rate your Professor

- https://www.courseexp.umd.edu/
- **If response rate reaches 80% for every section...**
- **by Sat 09-May-2026 11:59pm...**
- **I will reveal a Final Exam Question**
- No answers but public discussion welcome

N-Body Simulations

Several types of Physics and Chemistry systems are simulated in the following way

1. N particles exist in some space
2. A **Force Field** is present which describes attraction/repulsion between particles (and possibly external forces)
3. Particles have their positions initialized in the space
4. Forces acting on each particle are calculated using the Force Field and the current state (position/velocity) of the particles
5. The positions of each particle are adjusted by a small amount based on forces acting on them
6. Repeat 4/5 until bored

In Astrophysics, particles are usually celestial bodies (planets, stars, galaxies) and Force Field is Gravity.

Exercise: Sample N-Body

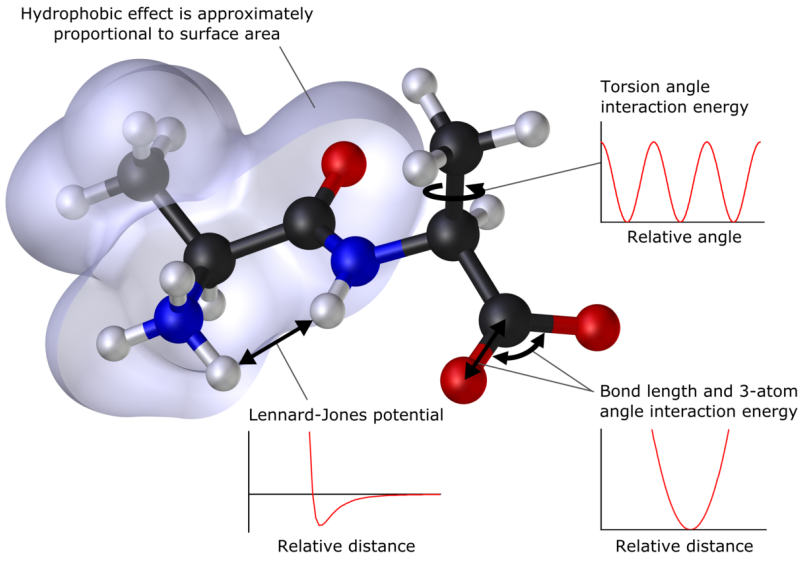
```
# calculate forces for new accelaration
for t in range(0,maxtime):
  for i=0 to N-1:
    acc[i].x = 0
    acc[i].y = 0
    for j=0 to N-1:
      if i==j: continue
      dx = pos[j].x - pos[i].x
      dy = pos[j].y - pos[i].y
      inv_r3 = (dx*dx + dy*dy)**(-1.5);
      acc[i].x +=  G * (dx * inv_r3) * mass[j];
      acc[i].y +=  G * (dy * inv_r3) * mass[j];

  # calculate new position
  for i=0 to N-1:
    vel[i].x += acc[i].x
    vel[i].y += acc[i].y
    pos[i].x += vel[i].x * dt
    pos[i].y += vel[i].y * dt
```

- Code represents (very rough) N-body simulation in 2D
- What opportunities do you see for parallelism?

Molecular Dynamics

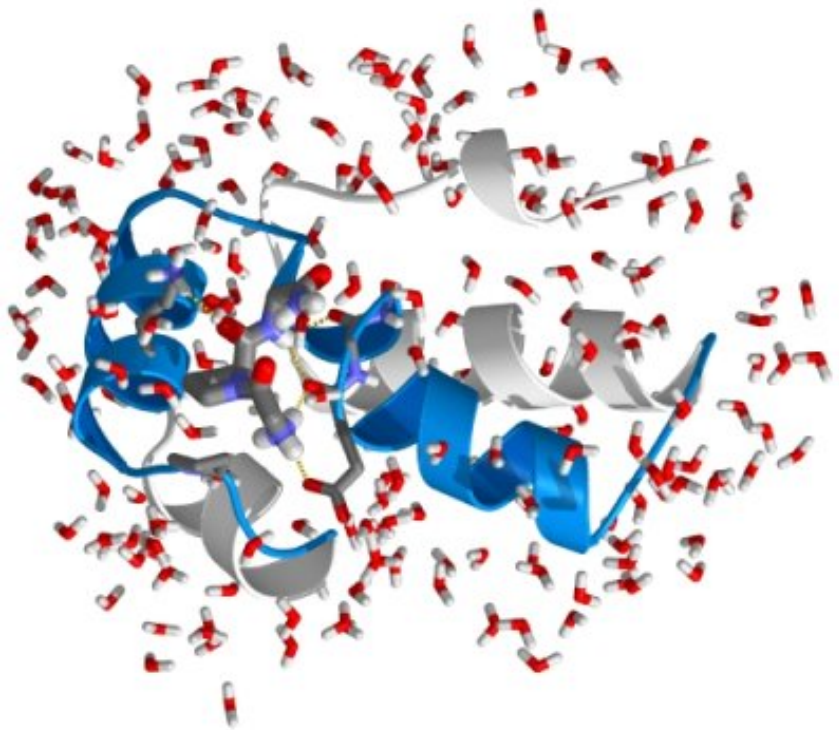
- Chemistry simulations can use a similar approximation to N-body but are focused on microscopic systems: particles are Atoms and Force Field is electrical interactions
- **Bonds** between atoms which act like springs which stretch/compress and allow twisting
- **Van der Waals** forces between non-bonded atoms which exerts attractive/repulsive forces



Source: Wikip Molecular Mechanics

Protein Folding

- Protein Folding is one heavily studies area of Molecular Dynamics
- Through experimental techniques / genetics can determine the bonded atoms a protein: 1,000-25,000 atoms with water around it
- Proteins are a large linear molecule that "folds" due to interactions with surrounding water + self attraction



Source: Intro to protein simulations

Segue to Machine Learning

- Protein Folding is a tough physic simulation
- Has led to various approximations, heuristic, computational approaches. Notably, recent Deep Learning approaches...

AlphaFold

From Wikipedia, the free encyclopedia

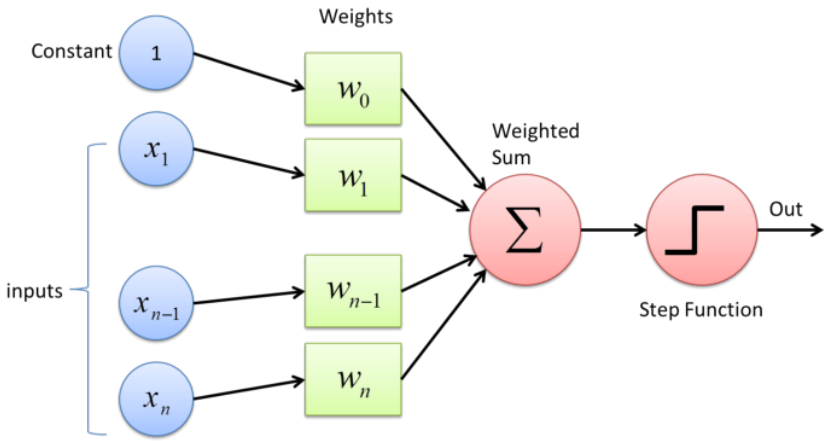
AlphaFold is an artificial intelligence (AI) program developed by Alphabets's/Google's DeepMind which performs predictions of protein structure.[1] The program is designed as a deep learning system.[2]

AlphaFold AI software has had two major versions. A team of researchers that used AlphaFold 1 (2018) placed first in the overall rankings of the 13th Critical Assessment of Techniques for Protein Structure Prediction (CASP) in December 2018. The program was particularly successful at predicting the most accurate structure for targets rated as the most difficult by the competition organisers, where no existing template structures were available from proteins with a partially similar sequence.

A team that used AlphaFold 2 (2020) repeated the placement in the CASP competition in November 2020.[3] The team achieved a level of accuracy much higher than any other group.[2] It scored above 90 for around two-thirds of the proteins in CASP's global distance test (GDT), a test that measures the degree to which a computational program predicted structure is similar to the lab experiment determined structure, with 100 being a complete match, within the distance cutoff used for calculating GDT.[2][4]

Perceptrons

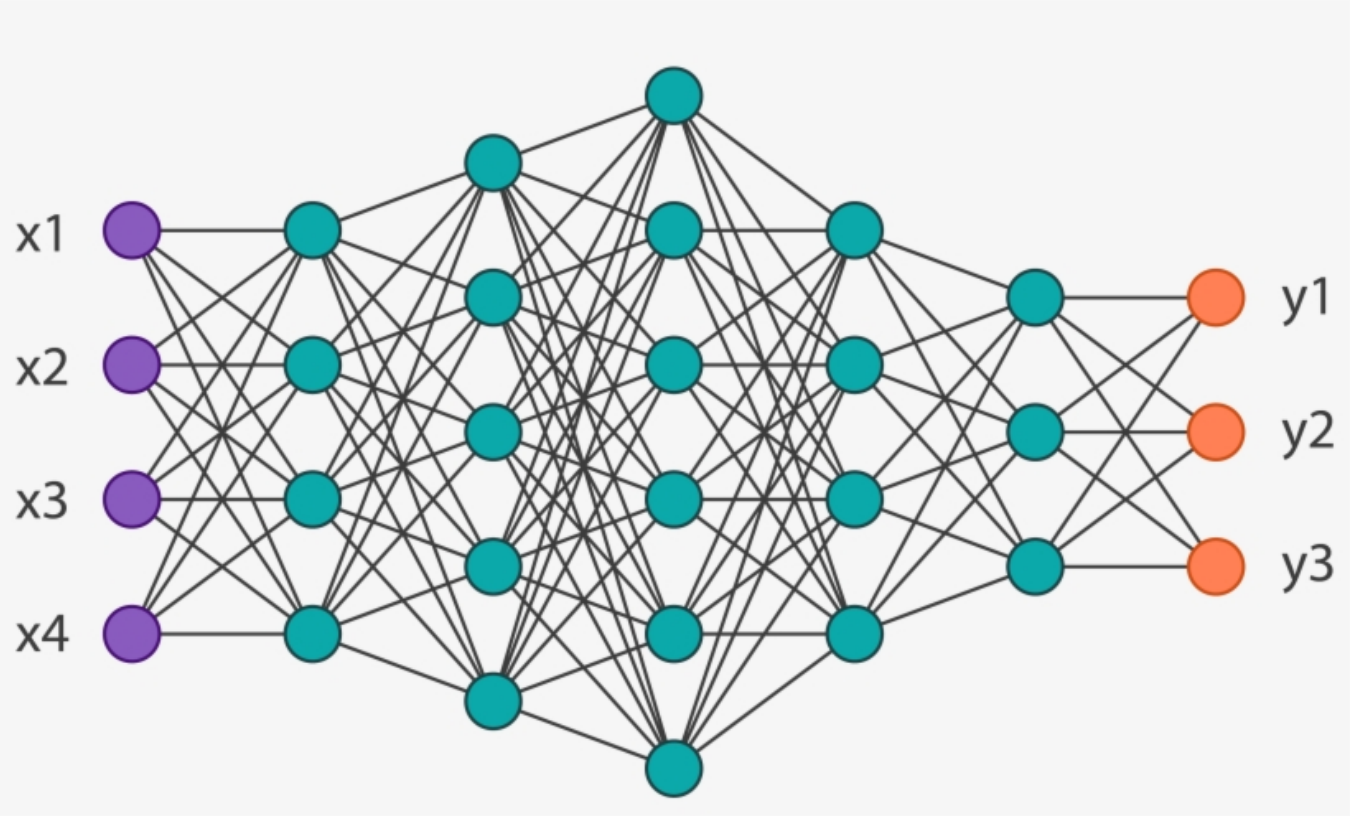
- Neural networks are comprised of a series of **Perceptrons**
- Input data vector is multiplied by a Weight vector and summed
- Basically a dot product followed by a (somewhat) arbitrary Activation function
- In a Single Perceptron, there is a vector of weights that are learned through gradient descent to predict an output value



Source: "What the Hell is a Perceptron" by Towards Data Science

- Will focus on Parallelism in computing outputs for neural networks - also parallelism in **backpropagation** to learn weights

Exercise: Neural Networks and Parallelism

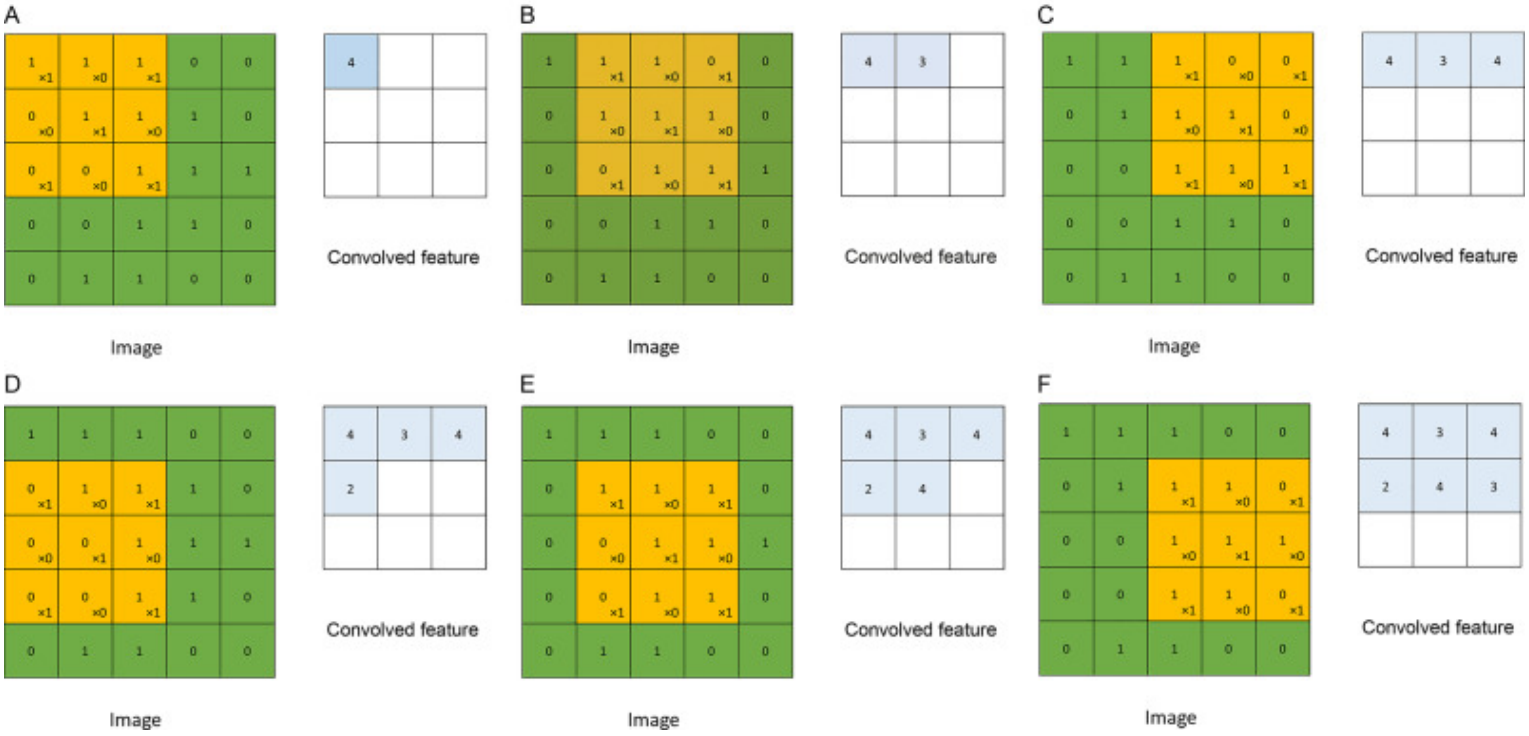


- Neural nets are layers of Perceptrons connected in some pattern: outputs in one layer become inputs in another
- Where can one find parallelism when calculating inputs to outputs in a neural net?

Examine: Pytorch Source Code

```
// pytorch/aten/src/ATen/native/cuda/Activation.cu
template <typename scalar_t>
inline void _rrelu_with_noise_cuda_train(...)
{  ...;
  rrelu_with_noise_cuda_kernel<scalar_t, 2><<<grid, block, 0, stream>>>(
      numel,
      rng_engine_inputs,
      output_data,
      input_data,
      ...);
}
__global__ void rrelu_with_noise_cuda_kernel(
    int numel,
    PhiloxCudaState philox_args,
    scalar_t* output,
    scalar_t* input,
    ...)
{ ...;
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  int grid_stride = blockDim.x * gridDim.x * unroll_factor;
  ...;
  for (int linear_index = idx;
       linear_index < rounded_size;
       linear_index += grid_stride)
  {
    ...;
  }
}
```

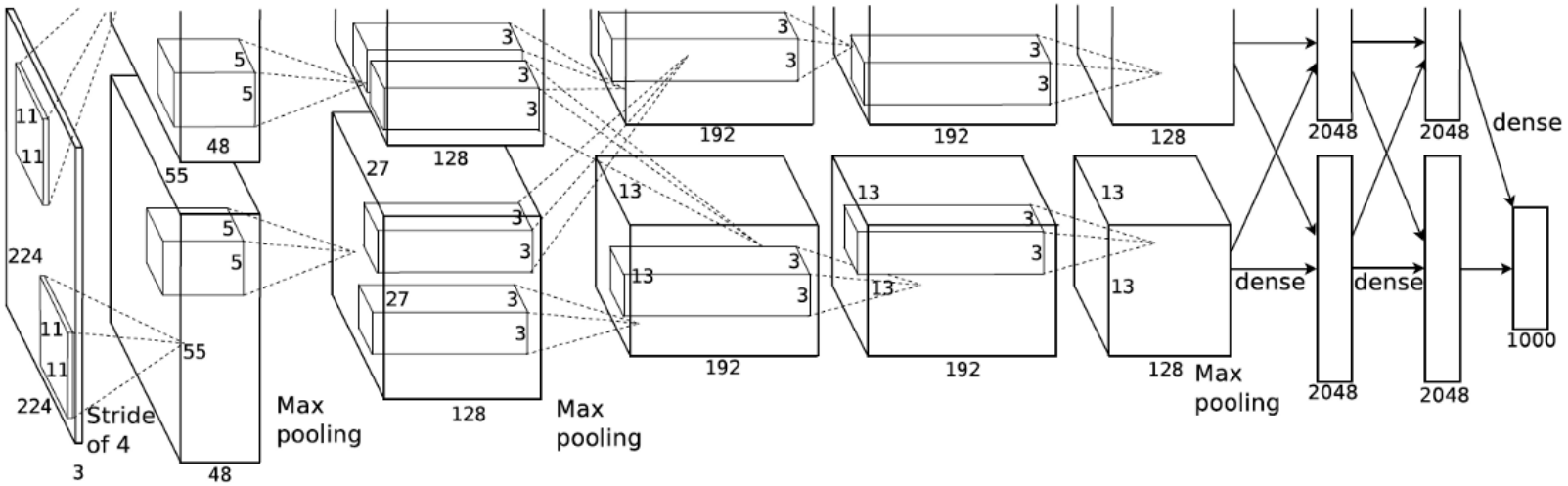
Convolution Neural Networks: Rife with Parallelism



Deep Neural Networks for Natural Language Processing Ehsan Fathi, Babak Maleki Shoja, Handbook of Statistics 2018

- In image processing, a **Convolution** is the application of a transformation matrix to an input image to produce a new matrix - repeated matrix inner product (Frobenius)
- Serve as the basis of Convolution Neural Networks where
  - Transformation reduces the size of the original image
  - Output matrix is used as input to deeper layers

Example CNN: AlexNet



ImageNet Classification with Deep Convolutional Neural Networks by Krizhevsky et al, Advances in Neural Information Processing Systems 25 (NIPS 2012)

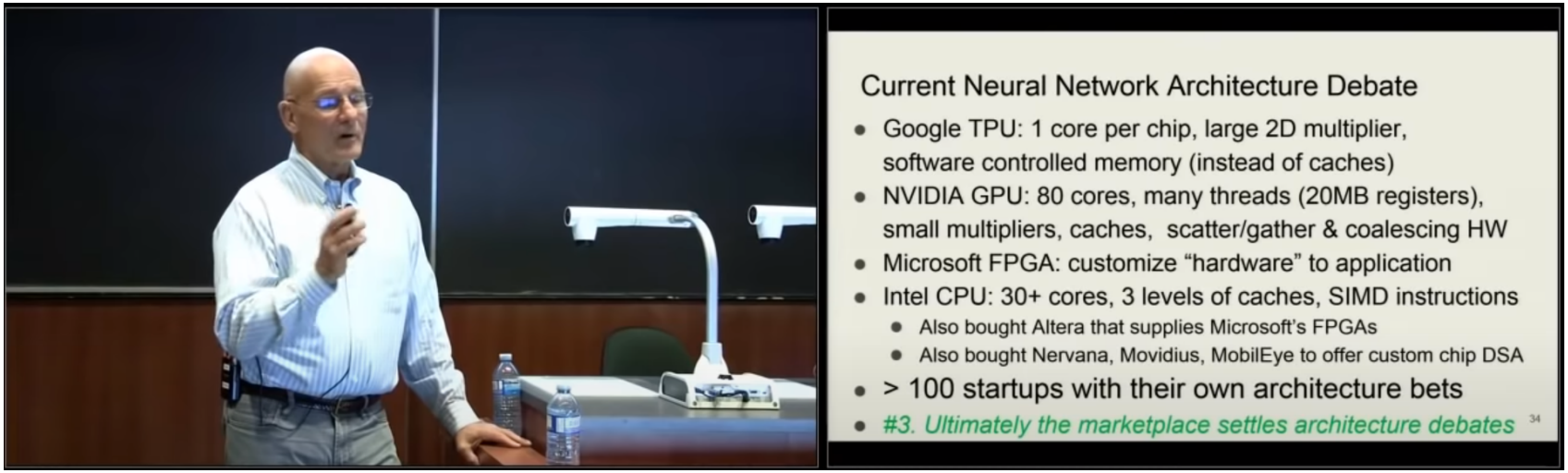
- First Convolution layer has 96 "filters" applied across images in 11x11 tiles
- 11x11 weights are "learned" and applied to image tile to produce 1 internal output - not fully connected
- Later layers have additional convolutions
- Ends with traditional Fully Connected "dense" layers to predict 1 of 1000 classes of images (dog, cat, car, etc.)
- Trained on 2 GPUs via CUDA to exploit parallelism

GPU Acceleration of Neural Nets

- Standard Neural Net layers boil down to a Matrix-Vector multiply then applying a function to each output vector element
- CNNs involve matrix-matrix multiplication followed by activation functions
- Transformer Architecture networks are rife with explicit matrix multiply operations
- Neural nets can be done on distributed machines but it is more common to use single machines with multithreading
- GPUs fit the problem extremely nicely and are often used to accelerate Neural Network training and predictions
- Popular packages for neural networks like
  - TensorFlow (by Google)
  - Pytorch (based on Facebook's Torch library)

  all include GPU support for accelerating neural network operations

"A Cornucopia of Architectures"



Source: New Golden Age for Computer Architecture by David Patterson, 2019 (Youtube)

- In addition to GPUs, a wide variety of new computer architecture endeavors are specifically targeting Neural Network applications
- Application Specific Integrated Circuits (ASICs) to exploit parallel structure in training and inference
- Example: Google's Tensor Processing Unit which *"transfer data to or from the host, perform matrix multiplications or convolutions, and apply activation functions."*

Cryptocurrency Mining

- A big topic, will look at one specific aspect: **Proof of Work** in Bitcoin
- Alice sends Bob 2 Bitcoins, a **transaction**
- Alice signs transaction with a private key to verify it is legit
- This transaction + 5 other transaction get grouped into a **block** of transactions:

  `[(A-2, B+2), (C-0.5, D+0.5), (E-3,A+1,B+1,C+1),....]`
- **Miners** provide record keeping for the block as a service by calculating the **Proof of Work** (POW) for the block
- Miners **compete** to find the PoW: doing so gets them transaction fees from Alice and others on the transaction AND earns them a small reward - newly minted Bitcoins

Proof of Work (PoW)

See: https://www.blockchain.com/btc/block/660000

- Block has transaction data in some format
- Block also contains hash of previous block (the "chain")
- Proof of Work for the block is a cryptographic hash of
  - The block data
  - A unique identifier/key for the miner
  - A **nonce**, number that needs to be found

  Such that the hash value has N leading 0's
- Example:

  `0000000000000000008eddcaf078f12c69a439dde30dbb5aac3d9d94e9c18f6`
- Bitcoin adjusts N every 14 days so that miners can find PoW for 1 block every 10 minutes

Basic Approach for Proof of Work

```
int find_nonce(int my_id,
               int *blockdata[], int len,
               int difficulty)
{
  for(long nonce=1; ; nonce++){
    long hash = hash_data(nonce, my_id, blockdata, len);
    int lead_zeros = count_lead_zeros(hash);
    if(lead_zeros >= difficulty){
      return nonce;
    }
  }
  return 0;
}
```

Where is the parallelism here?

Bitcoin has Issues

The New York Times

Bitcoin Uses More Electricity Than Many Countries. How Is That Possible?

In 2009, you could mine one Bitcoin using a setup like this in your living room.

↓

↓

Amount of household electricity required to mine one coin: **a few seconds' worth**. Bitcoin's value: **basically nothing**.

Today, you'd need a room full of specialized machines, each costing thousands of dollars.

↓

↓

Amount of household electricity required: **9 years' worth**. (Put in terms of a typical home electricity bill: **about $12,500**.) Value

Source: NYTimes 9/3/2021

- Miners receive transaction fees + reward for keeping the block chain publicly verified
- Requiring a certain amount of work to verify blocks means as more miners enter the scheme, more competition results and more power is spent
- Leads to explosion of power usage just to brute force a bunch of hashes
- Some schemes have been suggested for alternate system designs but little movement yet for the main cryptocurrencies